

# Travel Booking Automation Framework

---

## Project Documentation

### GitHub Repository

Repository Link:

[ust-sdet-training/SDET: Repository for SDET training batch](#)

---

### 1. Project Objective

This project was developed to automate a Travel Booking application by covering the major testing layers: UI Automation, API Automation and Database readiness. The objective was to build a reusable, maintainable framework that follows good automation practices and supports future CI/CD integration.

### 2. Technology Stack

| Category          | Technology              |
|-------------------|-------------------------|
| UI Automation     | Playwright + TypeScript |
| API Automation    | Java + Rest Assured     |
| Testing Framework | JUnit 5                 |
| Build Tool        | Maven                   |
| IDE               | VS Code & IntelliJ IDEA |
| Logging           | Logback                 |
| Environment       | Dotenv (.env)           |
| Version Control   | Git & GitHub            |

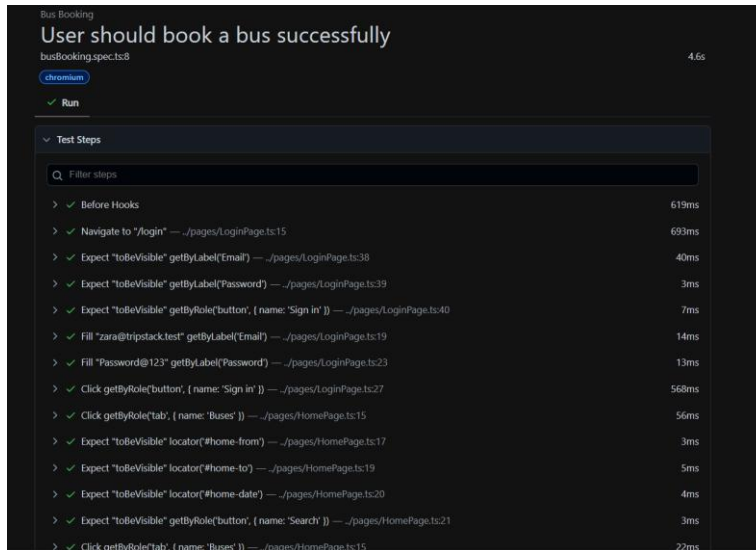
### 3. UI Automation

The UI framework was designed using the Page Object Model (POM) to improve maintainability and code reuse. Reusable page classes, fixtures and helper utilities were created so that new test cases can be added with minimal code duplication.

- Designed a scalable Playwright project structure.
- Implemented Page Object Model (POM).
- Created reusable page classes and locators.
- Used fixtures to reduce repeated setup code.
- Configured environment variables using .env.

- Integrated logging for execution tracking.
- Generated HTML reports with screenshots/traces.
- Executed end-to-end functional scenarios.

Screenshot Placeholder:



## 4. API Automation

A reusable Rest Assured framework was developed using a BaseClient architecture. Authentication, search, booking and payment modules were separated into dedicated client classes to improve readability and reuse.

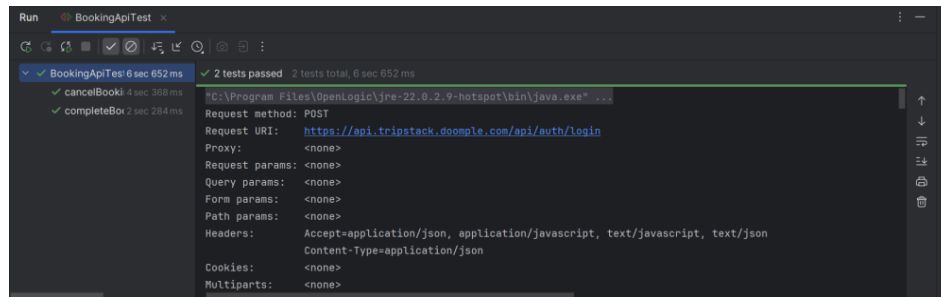
- Implemented reusable BaseClient for common HTTP methods.
- Implemented AuthClient, SearchClient, BookingClient and PaymentClient.
- Configured reusable Request Specifications.
- Implemented JWT token authentication.
- Created request/response models.
- Automated Login API.
- Automated Current User API.
- Automated Search Buses API.
- Automated Seat Map API.
- Automated Hold Booking API.
- Automated Payment API.
- Automated Confirm Booking API.
- Automated Cancel Booking API.
- Automated Booking Retrieval APIs.
- Validated response status codes and response bodies.

- Handled business scenarios such as Seat Conflict and invalid requests.

Booking Workflow:

Login → Search Bus → Select Seat → Hold Booking → Payment → Confirm Booking → Booking Completed

Screenshot:



## 5. Database

A separate database module was prepared to support future database validations. The project structure is ready for executing SQL queries to validate data created through UI/API execution.

- Prepared dedicated database project structure.
- Designed reusable database configuration.
- Planned validation of booking data after execution.
- Ready for future integration with automated SQL validations.

## 6. Challenges Faced

- Understanding the API contract using OpenAPI.
- Handling JWT authentication.
- Building reusable API clients.
- Debugging request payload mismatches.
- Handling seat conflict scenarios.
- Designing maintainable Page Object Model.
- Managing environment configuration using .env.

## 7. Key Learnings

- Framework design for UI and API automation.
- Reusable code using Page Object Model and BaseClient architecture.
- JWT-based authentication flow.
- API request/response validation.

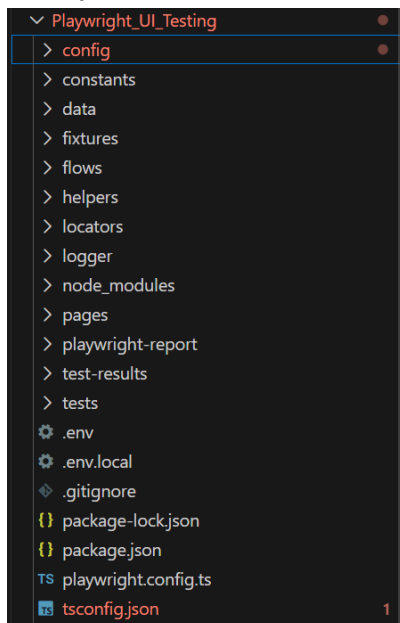
- Debugging HTTP requests and responses.
- Project organization using industry-standard folder structure.
- Writing scalable and maintainable automation code.

## 8. Future Enhancements

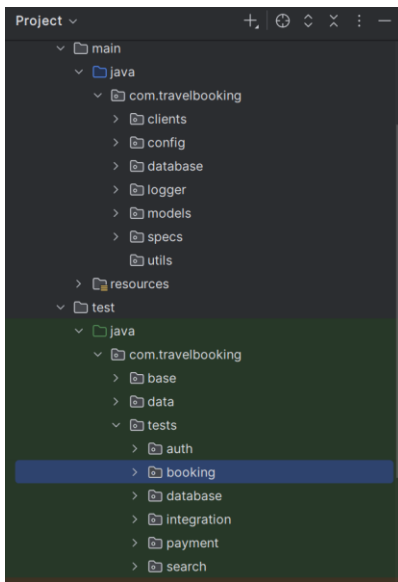
- GitHub Actions CI/CD integration.
- Allure Reporting.
- Dynamic seat selection for booking flow.
- Database validation implementation.
- Parallel execution.
- Jenkins integration.

## 9. Screenshots

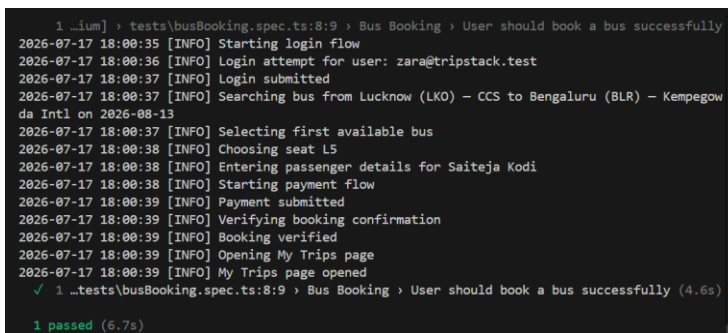
- UI Project Structure



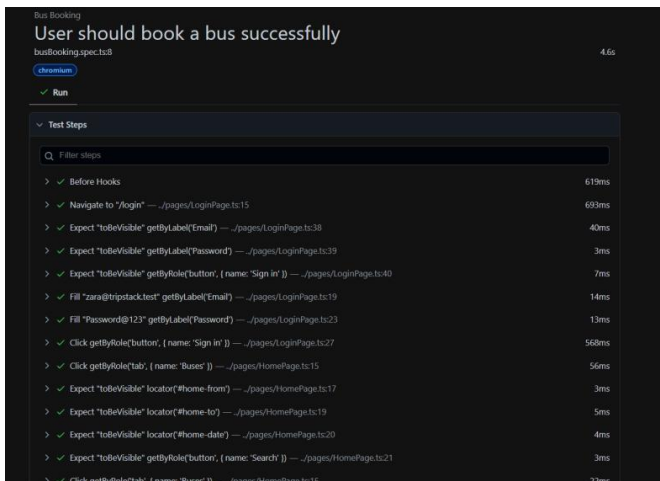
- API and DB Project Structure



- Successful UI Test Execution



- Playwright HTML Report



- Successful API Execution

